

Standard Operating Procedure (SOP)

Eco Sudar Bioenergy LLP — Admin Dashboard Development & Deployment

Document ID: SOP-ECOSUDAR-ADMIN-001

Version: 1.0

Effective Date: May 2026

Prepared By: Process AI / EcoSudar Engineering

Classification: Internal — Confidential

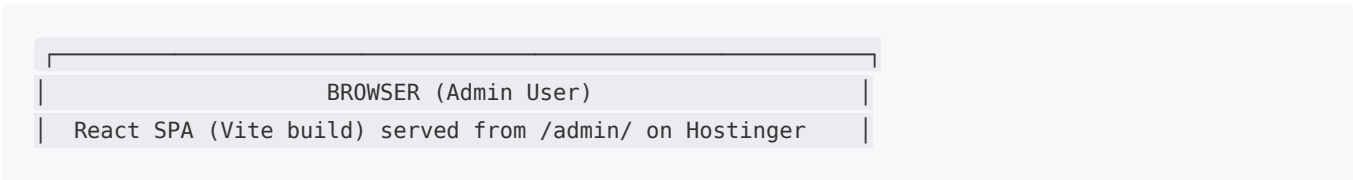
1. Purpose

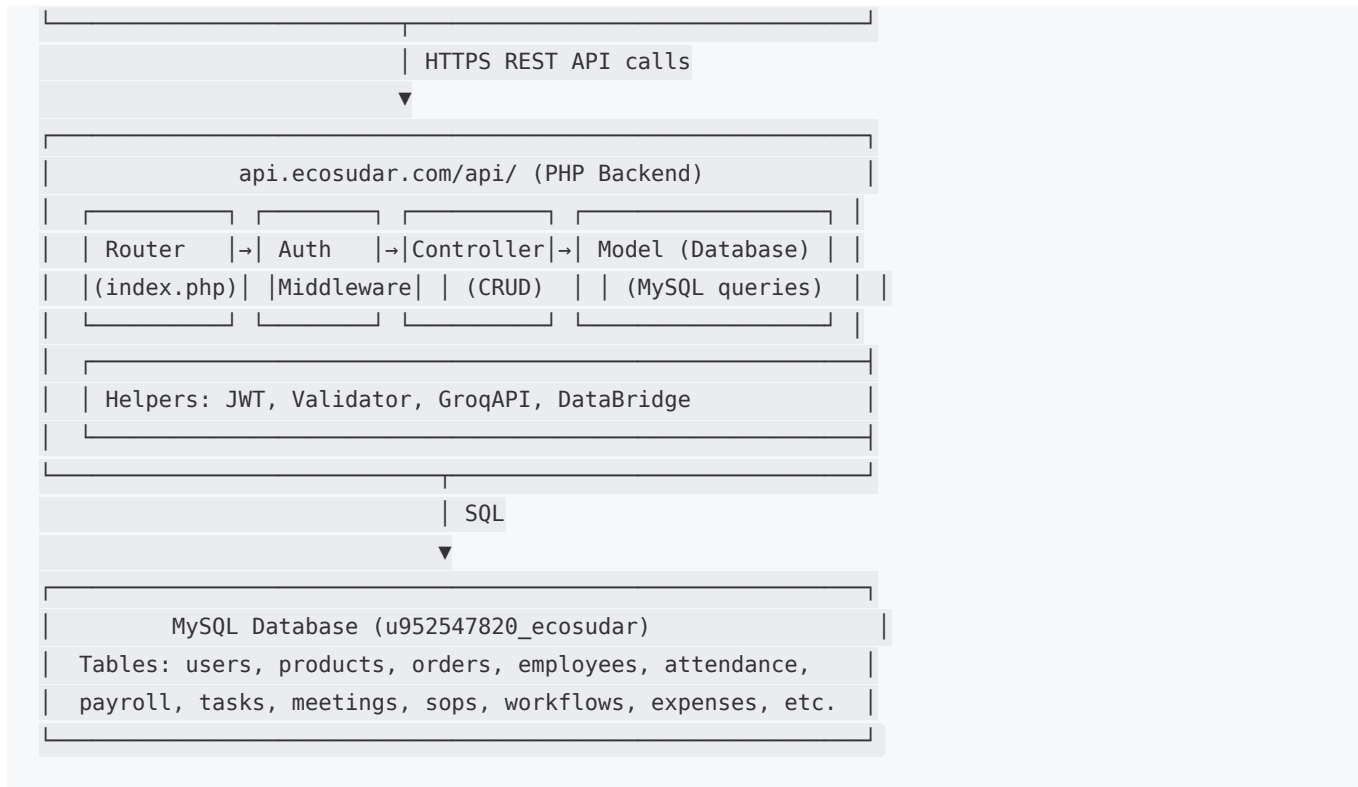
This SOP documents the end-to-end process for developing, deploying, and maintaining the **Eco Sudar Admin Dashboard** — a full-stack web application used by Eco Sudar Bioenergy LLP to manage customers, orders, invoices, employees, attendance, payroll, tasks, expenses, and AI-powered insights.

2. Scope

Area	Coverage
Frontend	React + TypeScript + Vite + ShadCN UI + TailwindCSS
Backend API	Custom PHP REST API (no framework)
Database	MySQL on Hostinger shared hosting
Hosting	Hostinger PHP/HTML server (<code>api.ecosudar.com</code>)
Deployment	Git → GitHub → Hostinger Git auto-deploy
AI Integration	Groq LLaMA + Google Gemini APIs

3. Architecture Overview





3.1 Directory Structure

```
api-EcoSudar/
├── .env                # Environment config (DB, JWT, API keys)
├── .htaccess           # Root Apache rewrite rules
├── admin/              # Built frontend (production)
├── api/                # PHP Backend
│   ├── index.php       # Entry point, CORS, routes
│   ├── config/         # app.php, database.php
│   ├── core/           # Database, Request, Response, Router
│   ├── middleware/     # AuthMiddleware, AdminMiddleware
│   ├── controllers/    # Public API controllers
│   │   └── admin/      # Admin-only controllers (19 controllers)
│   ├── models/         # Data models (14 models)
│   ├── helpers/        # JWT, Validator, GroqAPI, GroqClient
│   ├── uploads/        # Employee photos, insurance PDFs
│   └── .htaccess        # API-level rewrite rules
├── database/           # SQL schema + migrations
├── eco-sudar-control/  # Frontend source (React/Vite)
│   ├── src/pages/      # 27 page components
│   ├── src/components/ # Reusable UI components (ShadCN)
│   ├── src/lib/api/    # API client, HR, client modules
│   └── src/contexts/   # Auth context
└── deploy-admin.sh     # Build + deploy script
```

4. Development Workflow — Step by Step

Phase 1: UI Design with Lovable AI

Tool: [Lovable.dev](#) (AI-powered React app builder)

Step	Action	Details
1.1	Create project on Lovable	Select "React + TypeScript + Vite + ShadCN + Tailwind" stack
1.2	Describe the dashboard	Provide prompts for each module (Dashboard, Employees, Orders, etc.)
1.3	Iterate on UI	Review generated screens, request changes via chat prompts
1.4	Review components	Ensure consistent design language, responsive layouts, dark mode
1.5	Export source code	Download the complete project or push to GitHub from Lovable

Output: A fully scaffolded React project with pages, components, routing, and mock data.

[!IMPORTANT] Lovable generates the UI with mock/in-memory data. The real backend integration happens in Phase 4.

Phase 2: Transfer Source Code to Development Environment

Step	Action	Command/Details
2.1	Clone from GitHub	<code>git clone https://github.com/processai2026-arch/eco-sudar-admin-og.git</code>
2.2	Open in VS Code	Open the <code>eco-sudar-control/</code> directory
2.3	Install dependencies	<code>cd eco-sudar-control && npm install</code>
2.4	Run locally	<code>npm run dev</code> → opens at <code>http://localhost:8080</code>
2.5	Verify all pages render	Navigate through sidebar links, confirm mock data displays

Key Config Files:

- `eco-sudar-control/.env` → Set `VITE_API_BASE_URL` and `VITE_USE MOCK_API`
- `eco-sudar-control/vite.config.ts` → Dev server port, proxy settings

Phase 3: Hostinger Server Setup

3.1 Hosting Account Setup

Step	Action
3.1.1	Purchase Hostinger Premium/Business plan with PHP + MySQL support
3.1.2	Add domain <code>api.ecosudar.com</code> and configure DNS (A record → Hostinger IP)
3.1.3	Enable SSL certificate (Let's Encrypt) via Hostinger hPanel

3.2 Database Setup

Step	Action
3.2.1	Create MySQL database via hPanel → Databases → MySQL
3.2.2	Database name: <code>u952547820_ecosudar</code>
3.2.3	Create DB user with same name, set password
3.2.4	Import base schema: Upload <code>database/u952547820_ecosudar.sql</code> via phpMyAdmin
3.2.5	Run migrations from <code>database/migrations/</code> folder as needed

Key Tables:

Table	Purpose
<code>users</code>	Admin and customer accounts
<code>products</code>	Product catalog with size configurations
<code>orders</code>	Customer orders with status tracking
<code>employees</code>	HR records with photo, insurance, qualifications
<code>attendance</code>	Daily check-in/check-out with shift support
<code>payroll</code>	Monthly salary processing with PF/ESI/deductions
<code>tasks</code>	Task management with assignments and comments
<code>meetings</code>	Meeting scheduler with attendees
<code>sops</code>	Standard operating procedures
<code>workflows</code>	Multi-step workflow engine
<code>expenses</code>	Expense tracking with categories
<code>invoices</code>	Invoice generation (regular + GST)

3.3 File Structure on Server

<code>public_html/</code>	<code># Hostinger document root</code>
<code>├ .htaccess</code>	<code># Root rewrite rules</code>

```
├── admin/                                # Built React SPA
│   ├── index.html
│   └── assets/                          # JS, CSS, images
├── api/                                # PHP backend
│   ├── index.php                       # API entry point
│   ├── .htaccess                      # API rewrite rules
│   ├── config/
│   ├── core/
│   ├── controllers/
│   ├── models/
│   ├── middleware/
│   ├── helpers/
│   └── uploads/                        # File uploads directory
└── .env                               # Environment configuration
```

3.4 Environment Configuration (.env)

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=u952547820_ecosudar
DB_USER=u952547820_ecosudar
DB_PASS=<secure_password>

JWT_SECRET=<64_char_random_hex_string>

CORS_ORIGIN=https://api.ecosudar.com
```

Phase 4: Backend API Implementation

4.1 Architecture Pattern

The API uses a **custom MVC pattern** (no framework):

```
Request → .htaccess rewrite → index.php → Router
      → Middleware (Auth/Admin) → Controller → Model → Database
      → Response (JSON)
```

Core Components:

Component	File	Role
Router	core/Router.php	URL pattern matching with {param} support
Request	core/Request.php	Parse body, headers, query params, bearer token
Response	core/Response.php	JSON response with status codes
Database	core/Database.php	PDO wrapper with prepared statements

Component	File	Role
JWT	helpers/JWT.php	Token generation, validation, refresh
AuthMiddleware	middleware/AuthMiddleware.php	Bearer token validation
AdminMiddleware	middleware/AdminMiddleware.php	Admin role check

4.2 API Route Groups

Group	Prefix	Auth	Controllers
Auth	/auth/*	Public	AuthController
Users	/users/*	Bearer	UserController
Products	/products/*	Public read	ProductController
Orders	/orders/*	Bearer	OrderController
Statistics	/statistics/*	Bearer	StatisticsController
Admin Users	/admin/users/*	Admin	AdminUserController
Admin Products	/admin/products/*	Admin	AdminProductController
Admin Invoices	/admin/invoices/*	Admin	AdminInvoiceController
Admin Expenses	/admin/expenses/*	Admin	AdminExpenseController
Admin Finance	/admin/finance/*	Admin	AdminFinanceController
Admin Employees	/admin/employees/*	Admin	AdminEmployeeController
Admin Attendance	/admin/attendance/*	Admin	AdminAttendanceController
Admin Payroll	/admin/payroll/*	Admin	AdminPayrollController
Admin Tasks	/admin/tasks/*	Admin	AdminTaskController
Admin Meetings	/admin/meetings/*	Admin	AdminMeetingController
Admin SOPs	/admin/sops/*	Admin	AdminSopController
Admin Workflows	/admin/workflows/*	Admin	AdminWorkflowController
Chat (AI)	/chat/*	Optional	ChatController

4.3 Authentication Flow

1. POST /auth/login → { email, password }
2. Server validates → Returns { accessToken, refreshToken }
3. Frontend stores tokens in localStorage (eco_admin_auth)
4. All API calls include: Authorization: Bearer <accessToken>

5. Token refresh: POST /auth/refresh → new access token
6. Logout: POST /auth/logout → token blacklisted in revoked_tokens table

4.4 AI Integration

Feature	API Provider	Model
AI Chat Assistant	Groq	LLaMA 3.3 70B
Business Insights	Groq	LLaMA 3.3 70B
Bill Extraction (OCR)	Google Gemini	Gemini 2.0 Flash

Phase 5: Frontend ↔ Backend Integration

Step	Action	Details
5.1	Set <code>VITE_USE MOCK_API=false</code>	Switch from mock data to real API
5.2	Set <code>VITE_API_BASE_URL</code>	Point to <code>https://api.ecosudar.com/api</code>
5.3	Update API client	<code>src/lib/api/client.ts</code> — centralized fetch with auth headers
5.4	Map API responses	Ensure frontend types match backend JSON responses
5.5	Test each module	Verify CRUD operations for all 27 pages

API Client Pattern (`client.ts`):

```
export async function apiFetch<T>(path: string, init: RequestInit = {}): Promise<T> {
  const token = getAuthToken(); // from localStorage
  const headers = { Authorization: `Bearer ${token}`, ...init.headers };
  const response = await fetch(`${BASE_URL}${path}`, { ...init, headers });
  if (!response.ok) throw new Error(data.message);
  return data;
}
```

Phase 6: Deployment to Production

6.1 Git-Based Deployment

Step	Command	Description
6.1.1	<code>npm run deploy:admin</code>	Build frontend + copy to <code>admin/</code> folder
6.1.2	<code>git add -A && git commit -m "message"</code>	Stage and commit all changes
6.1.3	<code>git push origin main</code>	Push to GitHub

Step	Command	Description
6.1.4	Hostinger auto-pulls	Git deployment configured in hPanel

6.2 Deploy Script (`deploy-admin.sh`)

```
#!/usr/bin/env bash
# 1. Build the Vite project: npm run build
# 2. Clear old admin/ contents (preserving .htaccess)
# 3. Copy dist/ → admin/
# Run: npm run deploy:admin
```

6.3 SFTP Deployment (Alternative)

For manual file uploads when Git deployment is unavailable:

Step	Action
6.3.1	Open FileZilla or VS Code SFTP extension
6.3.2	Connect: Host= <code>ftp.ecosudar.com</code> , Port=21, Protocol=FTP
6.3.3	Navigate to <code>public_html/</code>
6.3.4	Upload changed files to their respective directories
6.3.5	Verify: Open <code>https://api.ecosudar.com</code> and hard-refresh

6.4 Post-Deployment Checklist

- ☐ Hard-refresh browser (Ctrl+Shift+R) to clear cached assets
- ☐ Verify login works
- ☐ Check API health: `GET /api/products` returns 200
- ☐ Verify new features function correctly
- ☐ Check PHP error log: `api/error_log`

5. Frontend Module Inventory

#	Page	File	Key Features
1	Dashboard	<code>Dashboard.tsx</code>	KPI cards, charts, recent activity
2	Employees	<code>Employees.tsx</code>	CRUD, photo upload, ID card generation
3	Attendance	<code>Attendance.tsx</code>	QR scan, shifts, daily report
4	Payroll	<code>Payroll.tsx</code>	Salary processing, PF/ESI, payslips

#	Page	File	Key Features
5	Tasks	Tasks.tsx	Assignment, status, priority, comments
6	Orders	Orders.tsx	Order management, status tracking
7	Products	Products.tsx	Product catalog, size pricing
8	Invoices	Invoices.tsx	Invoice generation, PDF download
9	GST Invoicing	GstInvoicing.tsx	GST-compliant invoice generation
10	Expenses	Expenses.tsx	Expense tracking, AI bill extraction
11	Finance	Finance.tsx	P&L, financial ratios
12	Customers	Customers.tsx	Customer management
13	Dealers	Dealers.tsx	Dealer management
14	Meetings	Meetings.tsx	Meeting scheduler, attendees
15	SOPs	Sops.tsx	Standard operating procedures
16	Workflows	Workflows.tsx	Multi-step workflow engine
17	Reports	Reports.tsx	Data export and reporting
18	Insights	Insights.tsx	AI-powered business analytics
19	FAQ	FAQ.tsx	FAQ management
20	Settings	Settings.tsx	System configuration
21	Pending Approvals	PendingApprovals.tsx	User registration approvals
22	Quote Requests	QuoteRequests.tsx	Customer quote management
23	Queries	Queries.tsx	Customer inquiry management
24	Advance Register	AdvanceRegister.tsx	Employee salary advances
25	Login	Login.tsx	Authentication screen

6. Technology Stack Summary

Layer	Technology	Version
Frontend Framework	React	18.3
Build Tool	Vite	5.4
Language	TypeScript	5.8
UI Components	ShadCN/UI (Radix)	Latest

Layer	Technology	Version
Styling	TailwindCSS	3.4
Charts	Recharts	2.15
PDF Generation	jsPDF + AutoTable	4.2 / 5.0
QR Code	qrcode + html5-qrcode	1.5 / 2.3
Backend Language	PHP	8.x
Database	MySQL	8.x
Web Server	LiteSpeed (Hostinger)	—
AI (Chat/Insights)	Groq LLaMA 3.3 70B	—
AI (OCR)	Google Gemini 2.0 Flash	—
Version Control	Git + GitHub	—

7. Security Measures

Control	Implementation
Authentication	JWT Bearer tokens (access + refresh)
Password Storage	<code>password_hash()</code> with <code>bcrypt</code>
Token Revocation	<code>revoked_tokens</code> table on logout
CORS	Whitelist-based origin control
SQL Injection	PDO prepared statements throughout
XSS Prevention	<code>htmlspecialchars()</code> input sanitization
File Upload	MIME type validation, size limits, path traversal prevention
Rate Limiting	Per-IP rate limiting on auth endpoints
Admin Guard	Two-layer middleware: <code>AuthMiddleware</code> → <code>AdminMiddleware</code>

8. Troubleshooting

Issue	Solution
API returns 401	Check token expiry, re-login
CORS error in browser	Verify <code>CORS_ORIGIN</code> in <code>.env</code> includes the frontend domain

Issue	Solution
500 Internal Server Error	Check <code>api/error_log</code> for PHP errors
Photos not loading (403)	Use the PHP photo endpoint (<code>/admin/employees/{id}/photo</code>) instead of direct file URLs
Build fails	Run <code>npm install</code> first, check Node.js version (18+)
Database connection error	Verify <code>.env</code> credentials, check Hostinger DB is running

9. Revision History

Version	Date	Author	Changes
1.0	May 2026	Process AI	Initial SOP creation

End of Document